

Les Arbres de Décision

Classification et Régression par Partitionnement Récursif

- 1 Qu'est-ce qu'un Arbre de Décision ?
- 2 Comment faire une prédiction avec un arbre de décision ?
- 3 Comment décide-t-on ?
- 4 Estimer des probabilités pour les classes
- 5 Algorithme d'apprentissage : CART
- 6 Régularisation et Hyperparamètres
- 7 Régression

1. Qu'est-ce qu'un Arbre de Décision ?

Arbre de Décision

Un **Arbre de Décision** est un algorithme d'apprentissage supervisé utilisé pour :

- La **classification** (prédire une catégorie)
- La **régression** (prédire une valeur continue)

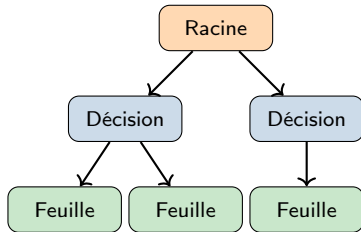
Intuition

C'est une structure de type **organigramme** où le modèle pose une série de questions pour affiner les possibilités jusqu'à prendre une décision finale.

Il imite la logique de prise de décision humaine.

Les Composants

- **Nœud Racine** : Point de départ, représente l'ensemble des données
- **Nœuds de Décision** : Sous-nœuds qui se divisent selon une caractéristique
- **Feuilles (Nœuds Terminaux)** : Nœuds finaux où une prédiction est faite
- **Branches** : Flèches reliant les nœuds (Vrai/Faux, Oui/Non)



Pourquoi les Arbres de Décision ?

Avantages Clés

- ➊ **Interprétabilité** : Facile à visualiser et expliquer aux non-experts
- ➋ **Pas de prétraitement** : Pas besoin de normaliser les données
- ➌ **Polyvalence** : Gère les données numériques ET catégorielles
- ➍ **Non-linéarité** : Capture des relations complexes sans transformation explicite

Le Fondement des Méthodes d'Ensemble

Les Arbres de Décision sont les briques de base des algorithmes puissants comme :

- **Random Forest** (Forêts Aléatoires)
- **Gradient Boosting** (XGBoost, LightGBM, CatBoost)

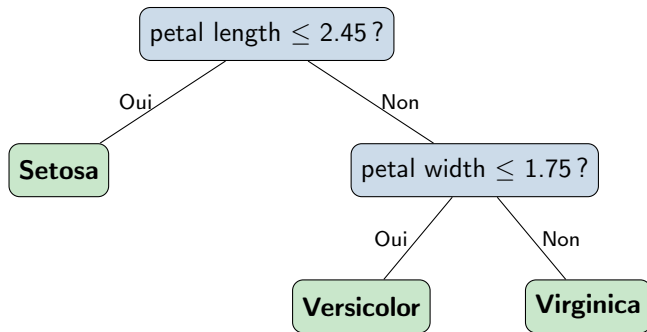
2. Comment faire une prédiction ?

Principe

Pour prédire la classe d'une nouvelle instance, on **parcourt l'arbre de la racine vers une feuille** en répondant aux questions à chaque nœud.

- ➊ **Départ** : Commencer au nœud racine
- ➋ **Test** : Évaluer la condition du nœud courant
- ➌ **Branchement** : Suivre la branche correspondante (Oui/Non, $\leq$ ou $>$)
- ➍ **Répéter** : Continuer jusqu'à atteindre une feuille
- ➎ **Prédiction** : La classe associée à la feuille est la prédiction

Exemple : Classification d'Iris



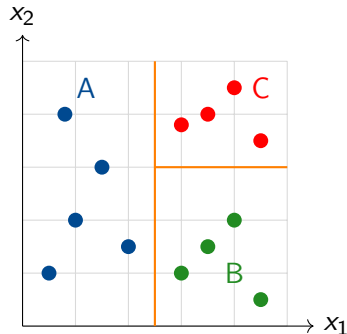
Nouvelle fleur : petal length = 4.5, petal width = 1.2 $\Rightarrow$ **Versicolor**

Caractéristique

Les arbres créent des **frontières de décision orthogonales** aux axes des caractéristiques.

Chaque division :

- Est **perpendiculaire** à un axe
- Utilise un **seuil unique**
- Divise l'espace en **rectangles**



3. Comment décide-t-on ?

Question Centrale

Comment l'arbre décide-t-il **quelle caractéristique** utiliser et **quel seuil** choisir pour diviser les données ?

L'Objectif : Trouver la division qui sépare les données en groupes les plus **homogènes** (purs) possible.

Mesures d'Impureté

Pour quantifier la pureté d'un nœud, on utilise :

- **Impureté de Gini** (défaut dans Scikit-Learn)
- **Entropie** (théorie de l'information)

Définition

L'**impureté de Gini** mesure la probabilité de mal classifier un élément choisi au hasard si on lui attribue une classe aléatoire selon la distribution du nœud.

$$\text{Gini}(S) = 1 - \sum_{k=1}^K p_k^2$$

où p_k est la proportion d'exemples de la classe k dans l'ensemble S .

Interprétation :

- Gini = 0 : Ensemble **pur**
- Gini = 0.5 : Impureté maximale (binaire)

Défaut Scikit-Learn

`criterion='gini'` est utilisé par défaut car plus rapide à calculer (pas de logarithme).

Exemple de Calcul : Gini

Scénario : Un nœud contient 100 emails : 60 spam et 40 non-spam

Gini du nœud :

$$\text{Gini} = 1 - \left(\frac{60}{100}\right)^2 - \left(\frac{40}{100}\right)^2 = 1 - 0.36 - 0.16 = \boxed{0.48}$$

Nœud pur (100 spam) :

$$\text{Gini} = 1 - 1^2 = 0$$

Nœud équilibré (50/50) :

$$\text{Gini} = 1 - 0.25 - 0.25 = 0.5$$

L'Entropie

Définition

L'**Entropie** mesure le degré d'incertitude ou de désordre dans un ensemble.

$$H(S) = - \sum_{k=1}^K p_k \log_2(p_k)$$

où p_k est la proportion d'exemples de la classe k .

Entropie = 0

Ensemble **pur** : tous les exemples appartiennent à une seule classe

Entropie = 1

Incertitude maximale : répartition 50/50 (cas binaire)

4. Estimer des probabilités

Le Besoin

Parfois, on ne veut pas seulement la **classe prédite**, mais aussi la **confiance** dans cette prédiction.

Solution

Chaque feuille stocke les **proportions des classes** des exemples d'entraînement qui y sont tombés.

Ces proportions servent d'estimation des probabilités.

Formule

Pour une instance arrivant dans une feuille contenant n exemples dont n_k de la classe k :

$$P(\text{classe} = k | \text{instance}) = \frac{n_k}{n}$$

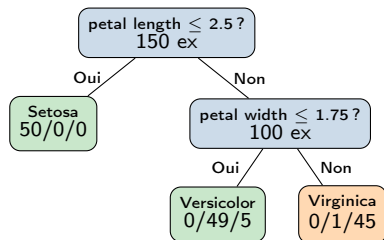
Exemple : Une feuille contient 40 exemples :

- 35 Virginica, 4 Versicolor, 1 Setosa

Probabilités estimées :

- $P(\text{Virginica}) = 35/40 = 87.5\%$
- $P(\text{Versicolor}) = 4/40 = 10\%$
- $P(\text{Setosa}) = 1/40 = 2.5\%$

Exemple Complet : Calcul des Probabilités



Nouvelle fleur :

petal length = 5.1

petal width = 1.9

Parcours :

- 1 $5.1 > 2.5 \Rightarrow$ droite
- 2 $1.9 > 1.75 \Rightarrow$ droite

Feuille atteinte : 46 ex
(0 Setosa, 1 Versi., 45 Virg.)

Probabilités :

- $P(\text{Set.}) = 0/46 = 0\%$
- $P(\text{Ver.}) = 1/46 = 2.2\%$
- $P(\text{Vir.}) = 45/46 = 97.8\%$

5. l'Algorithme d'Apprentissage : CART

CART : Classification And Regression Trees

Historique

Algorithme introduit par **Breiman et al. (1984)**.
C'est l'algorithme utilisé par Scikit-Learn.

Caractéristiques Clés

- Produit des **arbres binaires** (chaque nœud a exactement 2 enfants)
- Utilise **Gini** ou **Entropie** pour la classification
- Utilise la **variance** (MSE) pour la régression
- Approche **gloutonne** : optimisation locale à chaque nœud

Définition des Seuils Candidats

Les Données

Pour une caractéristique j , on observe n valeurs dans le nœud courant.
Après tri, on obtient les valeurs ordonnées : $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$

L'Ensemble des Seuils $\mathcal{T}_j$

On définit l'ensemble des seuils candidats pour la caractéristique j :

$$\mathcal{T}_j = \left\{ t_i = \frac{x_{(i)} + x_{(i+1)}}{2} \mid i \in \{1, \dots, n-1\} \right\}$$

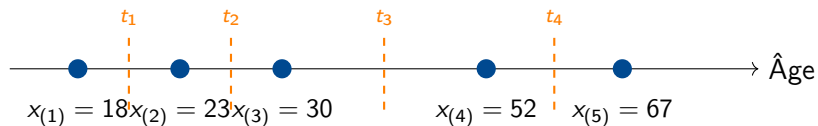
Ce sont les **milieux entre valeurs consécutives**.

Nombre de Seuils

$|\mathcal{T}_j| = n - 1$ seuils à tester pour chaque caractéristique.

Exemple : Construction de $\mathcal{T}_j$

Données : 5 exemples avec la caractéristique "Âge"



Ensemble des seuils :

$$\mathcal{T}_j = \left\{ \frac{18 + 23}{2}, \frac{23 + 30}{2}, \frac{30 + 52}{2}, \frac{52 + 67}{2} \right\} = \{20.5, 26.5, 41, 59.5\}$$

Pseudo-code : CART(S)

1. Si critère d'arrêt atteint $\Rightarrow$ créer une feuille
2. Sinon :
 - a. Pour chaque caractéristique j , pour chaque $t \in \mathcal{T}_j$: calculer l'impureté
 - b. Sélectionner (j^*, t^*) qui minimise l'impureté pondérée
 - c. Diviser : $S_L = \{x : x_{j^*} \leq t^*\}$ et $S_R = \{x : x_{j^*} > t^*\}$
 - d. Appeler CART(S_L) et CART(S_R)

L'arbre s'arrête de grandir si :

- Le nœud est **pur** (une seule classe)
- La **profondeur maximale** est atteinte (`max_depth`)
- Le nombre d'exemples est **insuffisant** pour diviser (`min_samples_split`)
- Le nombre d'exemples dans une feuille serait **trop faible** (`min_samples_leaf`)
- Le **gain** est inférieur au seuil (`min_impurity_decrease`)

Sans Restriction

CART construira un arbre jusqu'à ce que chaque feuille soit pure $\Rightarrow$ **Surapprentissage !**

6. Régularisation et Hyperparamètres

Problème Majeur

Sans contraintes, les arbres ont tendance à **mémoriser** les données d'entraînement plutôt que de généraliser.

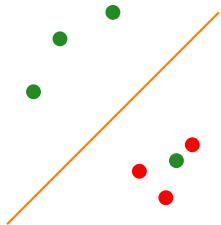
Un arbre non régularisé peut :

- Créer une feuille pour **chaque exemple** d'entraînement
- Atteindre **100% d'accuracy** sur le train set
- **Échouer complètement** sur de nouvelles données

Cause

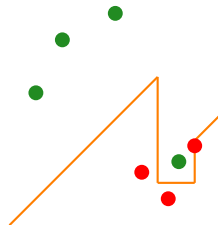
L'arbre capture le **bruit** en plus du signal.

Arbre Régularisé



Bonne généralisation

Arbre Surajusté



Apprend le bruit !

Pré-Élagage (Contraintes sur la croissance)

- `max_depth` : Profondeur maximale de l'arbre
- `min_samples_split` : Nb minimum d'exemples pour diviser un nœud
- `min_samples_leaf` : Nb minimum d'exemples dans une feuille
- `max_leaf_nodes` : Nombre maximum de feuilles
- `min_impurity_decrease` : Gain minimum pour effectuer une division

Conseil

Commencer par `max_depth` et `min_samples_leaf`.

Utiliser la **validation croisée** pour trouver les bonnes valeurs.

Post-Élagage : Cost-Complexity Pruning

Principe

- 1 Laisser l'arbre croître complètement
- 2 **Élaguer** les branches qui n'améliorent pas suffisamment la performance

Fonction de Coût

$$R_{\alpha}(T) = R(T) + \alpha \cdot |T|$$

où $R(T)$ est l'erreur de l'arbre, $|T|$ le nombre de feuilles, et α le paramètre de complexité.

Dans Scikit-Learn

Utilisez `ccp_alpha` pour contrôler le post-élagage.

7. Régression

Le Problème

Gini et Entropie sont conçues pour des **classes discrètes**.
Comment adapter les arbres pour prédire des valeurs continues ?

Solution : Utiliser la Variance (MSE)

Au lieu de mesurer la pureté des classes, on mesure la **dispersion** des valeurs cibles.

MSE d'un Ensemble

$$\text{MSE}(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y})^2$$

où $\bar{y}$ est la moyenne des valeurs cibles dans S .

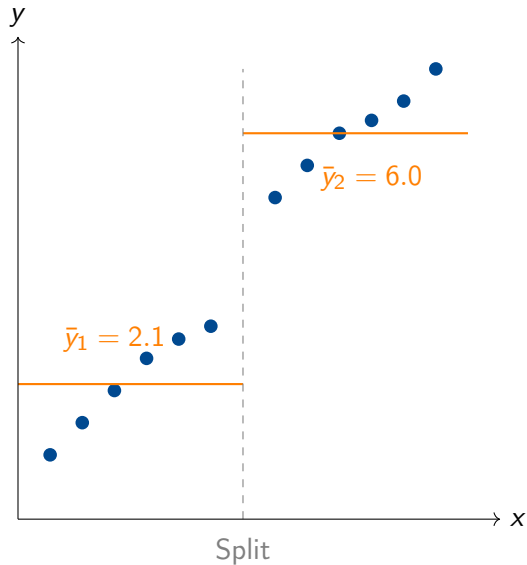
Objectif : Trouver la division qui **minimise** le MSE pondéré des sous-ensembles.

$$\text{MSE}_{split} = \frac{|S_L|}{|S|} \text{MSE}(S_L) + \frac{|S_R|}{|S|} \text{MSE}(S_R)$$

Prédiction dans une Feuille

La valeur prédite est la **moyenne** des valeurs cibles des exemples dans cette feuille.

Visualisation : Arbre de Régression



Prédictions Discontinues

- Les prédictions sont des **constantes par morceaux**
- Pas d'interpolation entre les régions
- Peut être grossier pour des fonctions lisses

Solutions

- Augmenter la profondeur (attention au surapprentissage)
- Utiliser des **ensembles d'arbres** (Random Forest, Gradient Boosting)
- Arbres avec régression linéaire dans les feuilles (M5, Cubist)

Résumé

- ❶ **Structure** : Les arbres posent des questions successives pour arriver à une prédiction
- ❷ **Prédiction** : Parcours de la racine vers une feuille en suivant les conditions
- ❸ **Division** : Gini et Entropie mesurent l'impureté pour choisir le meilleur split
- ❹ **Probabilités** : Les proportions dans les feuilles estiment les probabilités
- ❺ **CART** : Algorithme standard qui produit des arbres binaires
- ❻ **Régularisation** : Essentielle pour éviter le surapprentissage
- ❼ **Régression** : MSE remplace Gini, moyenne dans les feuilles

Des questions ?